

FIT 3182 2026 Assignment 3						
Marking Guide						
Marked By :						
Criterion	N: 0% to 49%	P: 50% to 59%	C: 60% to 69%	D: 70% to 79%	HD: 80% to 100%	Maximum Marks
1. Extension and implementation, backed by research articles	The work does not meaningfully extend the Assignment 2 pipeline, or the submitted implementation is largely non-functional. The extension may be superficial, unrelated to the AWAS streaming scenario, hardcoded, or not clearly based on any selected Section 3 theme. Research articles or technical sources are absent, irrelevant, or only listed without connection to the work.	A small working extension to the Assignment 2 pipeline is implemented. The extension is simple but identifiable, such as a basic dashboard, simple Docker setup, simple Spark ML experiment, basic Kafka Streams prototype, limited cloud or cloud-simulated deployment, or a small research-inspired modification. At least one relevant research article or credible technical source is mentioned, although the connection to the implementation may be basic.	The selected extension is implemented correctly for common cases and is clearly connected to the Assignment 2 pipeline. The implementation shows a reasonable understanding of the chosen theme, such as deployment, Spark ML, stream join modification, Kafka Streams, or dashboarding. Research or technical literature is used to motivate or explain the selected method.	The extension is well implemented, robust, and clearly integrated with the Assignment 2 pipeline. It handles realistic execution issues relevant to the selected theme, such as configuration, data flow, state handling, integration between components, error handling, or operational assumptions. Relevant research articles or technical literature are used to justify the chosen approach.	The extension is substantial, robust and technically impressive. It is deeply integrated with the Assignment 2 pipeline and demonstrates a strong understanding of big data streaming architecture. The implementation addresses realistic concerns such as state management, fault tolerance, idempotency, configuration, deployment boundaries, reproducibility, and operational limitations, as relevant. Research articles or high-quality technical literature are not merely cited; they are clearly used to shape the method, implementation, or evaluation.	2.0
2. Deployment and reproducibility	The solution cannot be reproduced by the teaching team. Setup instructions are missing, incorrect, or too incomplete to follow. Required configuration files, dependency details, deployment scripts, or execution steps are absent or unusable.	Basic setup instructions are provided. The solution can be run with some manual intervention, assumptions, or troubleshooting. Some dependency, configuration, or environment details may be incomplete.	The solution is mostly reproducible. README instructions are generally clear, and required dependencies, commands, configuration assumptions and execution sequence are documented. Some minor manual setup may still be needed.	The solution is reproducible with clear instructions and well-organised files. Deployment or cloud-simulated execution is properly documented using appropriate scripts, Docker/Docker Compose, configuration templates, cloud setup notes, or equivalent setup.	The solution is highly reproducible and well packaged. The teaching team can run the system using clear commands and configuration files. Deployment scripts, Docker/Docker Compose files, cloud setup notes, environment variables, dependency files, and data paths are well organised. The project structure is professional and easy to follow.	1.5
3. Innovation and technical complexity	The work shows little or no technical innovation beyond Assignment 2. Any claimed improvement is unclear, trivial, or not implemented. Design choices are not justified, and there is little evidence of technical understanding.	Innovation is limited. The solution mostly adds a small feature to Assignment 2, but it demonstrates some additional capability. Design choices are explained at a basic level.	The extension shows moderate technical effort. It is more than a superficial add-on, but may still rely on standard implementation patterns. Key design choices are explained, although justification may be somewhat descriptive rather than deeply analytical.	The work shows clear technical ambition. The extension involves a non-trivial design or implementation choice, such as modifying the join strategy, using a cloud deployment pattern, building an interactive operational dashboard, integrating ML into the streaming workflow, or replacing Spark with Kafka Streams in a meaningful way. Key design choices are justified with reference to technical trade-offs.	The work demonstrates clear innovation beyond Assignment 2. The extension is technically non-trivial and shows independent design thinking. Examples may include implementing or adapting a research based stream join algorithm, comparing alternative join strategies, designing a scalable cloud-simulated architecture, integrating predictive analytics meaningfully into the streaming workflow, or developing an interactive dashboard that supports operational decision making. The technical contribution is clearly explained and justified using relevant literature.	3.0
4. Performance analysis	Little or no attempt is made to evaluate performance, scalability, throughput, latency, resource use, state growth, complexity, or operational behaviour. Claims are unsupported by evidence.	Provides a minimal performance discussion or simple test. For example, the student may compare small and larger input batches, report basic runtime observations, or discuss expected scalability limitations without deeper evaluation.	Provides some meaningful performance or operational analysis. This may include basic measurements of runtime, throughput, latency, message volume, resource use, batch size effects, join behaviour, state size, or deployment limitations.	Provides meaningful evaluation with evidence. The student considers throughput, latency, data volume, resource use, scaling behaviour, bottlenecks, limitations, complexity, or operational trade-offs. The analysis goes beyond simply showing that the code runs.	Provides strong evidence based evaluation. The student performs meaningful experiments or analysis involving scalability, throughput, latency, resource usage, input volume, state growth, windowing behaviour, join cost, deployment constraints, complexity, or operational bottlenecks. Results are interpreted critically, with clear discussion of why they matter and what limitations remain.	1.5

5. Documentation, presentation and communication	README, slides, video, or demonstration are missing or very poor. The student does not clearly explain the method, implementation, results, or limitations. Camera is not turned on	README, slides and video are present and understandable. The explanation covers what was done, but may lack depth, polish, or critical reflection. The demonstration may be basic but shows that the extension works.	The README, slides and video explain the selected theme, architecture, implementation, results and basic research support clearly. The demonstration is understandable and shows the working application or extension.	The slides, video, and README are clear, professional, and technically informative. The student explains not only what was implemented, but also why key decisions were made, what results were obtained, and what trade-offs or limitations remain.	The slides, README and video are polished, concise and technically convincing. The presentation clearly communicates the problem, selected theme, architecture, implementation, research support, performance findings, results and limitations. The demonstration is smooth and shows clear ownership of the work.	2.0
--	---	---	--	--	---	-----